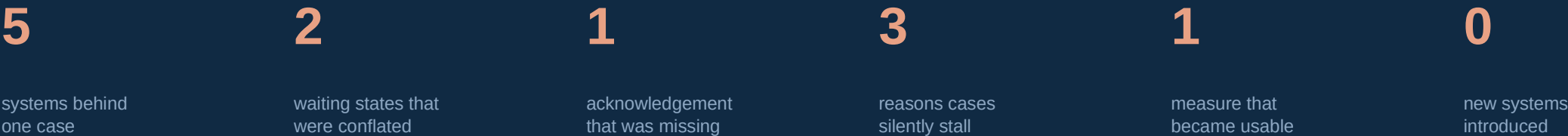


Kotak Mahindra Life

Case visibility across five systems — and the smallest fix that mattered most

A servicing case in flight across five systems is visible in all of them and understood in none. The work that changed the operation was not a dashboard; it was distinguishing a case waiting on the insurer from a case waiting on the customer, and telling the customer their case existed.



PROGRAMME CONTEXT · LIFE INSURANCE POLICY SERVICING OPERATIONS

Open case visibility and reverts · automated acknowledgement for manually created cases · integration across the servicing estate

Aditya Singh · Senior Manager — policy servicing programmes · 2024 and 2026

Vendor, platform and system names removed. Programme scope is from the delivery record; the state analysis and stall decomposition are design work.

EXECUTIVE SUMMARY

Turnaround was unmeasurable because two completely different waiting states were being counted as one — and the fix was smaller than anyone expected

Situation

Policy servicing cases move across five systems: the servicing platform, a rules engine, the policy administration system, the customer-facing policy manager and a reporting service.

Each system knows about the case. None holds the complete picture of where it actually is.

Complication

A case waiting for the insurer to act and a case waiting for the customer to send a document are entirely different operational states, with different owners and different remedies.

Counted together, the turnaround figure measures neither. Operations cannot tell whether a backlog is a capacity problem or a chasing problem, and no service-level commitment can honestly be made — because the number includes time the insurer does not control.

What changed

Separate the two states explicitly at the case level, so a revert to the customer stops the insurer's clock and starts a different one.

Then close the loop that made the second state so damaging: acknowledge every case at creation, including the manually raised ones that previously entered the estate silently. A customer who does not know a case exists cannot respond to a revert on it.

THE NUMBERS THAT MATTER

2

states, previously one

5

systems, one case view

1

clock the insurer owns

100%

of cases acknowledged

3

stall causes identified

1

measure now actionable

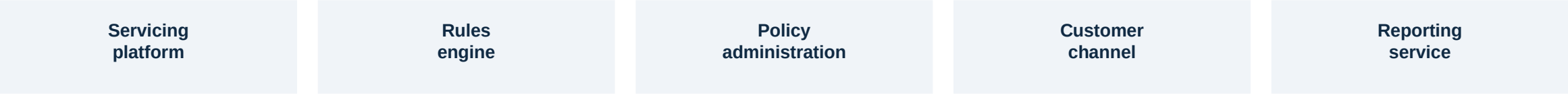
The judgement this argues: most servicing visibility problems are presented as reporting problems and solved with dashboards. This one was a state-modelling problem with a communication gap behind it, and no dashboard over the original model would have produced an actionable number.

The programmes and the estate come from the delivery record; the state model and the stall analysis are design work

DELIVERED SCOPE Programme context The servicing estate and the two programmes as delivered. • Open case visibility and reverts, 2024 • Automated acknowledgement, 2026 • Five systems in the servicing chain • Manually created case pathway • Existing reporting service • Customer-facing policy channel	DELIVERY RECORD My ownership Delivery lead on both programmes, including the state separation and the acknowledgement design. • Waiting-state separation design • Revert lifecycle and clock handling • Acknowledgement for manual cases • Cross-system case view • Reporting continuity • Requirement through production	METHOD ILLUSTRATION Reconstructed The design detail shown illustrates the approach taken. Specific definitions, categories and figures are reconstructed for presentation and are not quoted from the programme record. • Approach is from the engagement • Detail is reconstructed to illustrate it • Figures are directional, never measured • Confirm specifics before citing them • The method is what transfers • Ask me for the underlying record	OUT OF SCOPE Not claimed Excluded deliberately. • Any customer or policy data • Any case volume or service level • The insurer's operational metrics • Any confidential document • Measured business outcomes • Any underwriting decision
--	---	---	---

THE PROBLEM

Five systems each hold part of the truth — and the question operations actually needs answered is not stored anywhere



Each knows the case exists. Each holds a different fragment of its state.

The question	Where the answer lives	Why it could not be answered
Does a case exist for this customer?	Servicing platform	Manually created cases entered silently
What is it waiting for?	Split across platform and rules engine	No single field expressed the reason
Is that our delay or theirs?	Nowhere	Both were the same status value
Has the customer been told?	Nowhere reliably	Acknowledgement was manual and inconsistent
How long has it really taken us?	Reporting service	Included time waiting on the customer
What is the complete document set?	Split across three systems	No single view of completeness

Why this presents as a reporting problem and is not one

Operations asked for a dashboard, and the request was entirely reasonable — they could not see the estate. But a dashboard built over the existing model would have shown the same conflated status with better formatting, and would have been quietly abandoned within two quarters.

Source: Author's delivery record, policy servicing programmes, author's analysis.

Two clocks on one case — the insurer's clock stops when a revert is raised and starts again when the customer responds

STATE ONE

Waiting on us

- The insurer holds everything it needs and has work to do
- This is the only time that belongs in a service-level commitment
- Owned by operations, with a queue, a capacity model and an escalation path
- A backlog here is a resourcing or routing problem, and is fixable internally

STATE TWO

Waiting on the customer

- A revert has been raised and the case cannot progress without a response
- Excluded from the insurer's service-level clock, and reported separately
- Owned by a chasing cycle with defined contacts, intervals and a closure rule
- A backlog here is a communication problem, and no amount of capacity fixes it

What the separation enabled

- A turnaround figure that measures only what the insurer controls — and can therefore be committed to.
- A separate, visible reverts population with its own ageing and its own chasing cadence.
- The ability to tell a capacity problem from a chasing problem, which had previously been indistinguishable.

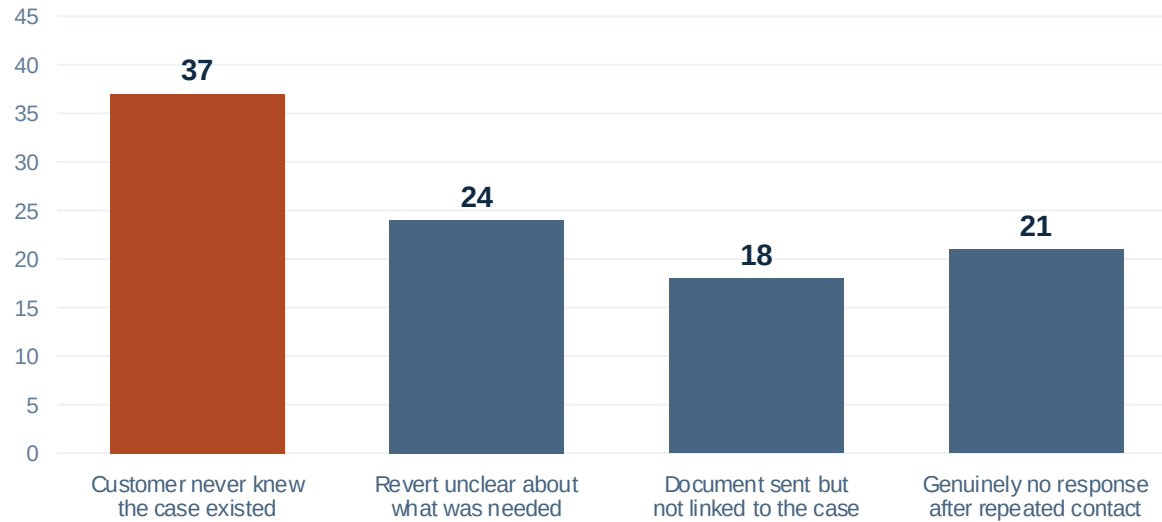
The rules that make it honest

- The clock stops only when a revert is genuinely raised and communicated — not when a case is parked.
- A revert raised and never sent does not stop the clock; the send event does.
- Reverts age, and an aged revert escalates rather than sitting indefinitely in a state nobody reports on.

The failure mode to design against

- A stoppable clock is an invitation to stop it. Without the send-event rule, reverts become a way to manage the metric.
- Reporting both populations side by side, always, is what prevents that.
- A falling turnaround alongside a rising reverts backlog is a signal, not a success.

Manually created cases entered the estate silently — so a customer could receive a revert on a case they never knew had been opened



How the gap arose

- Cases raised through the customer channel generated an acknowledgement automatically — the channel did it.
- Cases raised internally by an agent, from a call or an email, did not. The acknowledgement depended on the agent remembering.
- Both populations then behaved identically in the estate, so the difference was invisible in every report.
- A revert on an unacknowledged case reaches a customer with no context for it — and is frequently ignored as unsolicited.

Illustrative decomposition of cases stalled in the waiting-on-customer state

Why an acknowledgement email is the highest-return item in this deck

It is not an impressive artefact. It generated no dashboard, no model and no architecture diagram, and it took a fraction of the effort of the visibility work around it.

But the largest single cause of cases stalling was customers not knowing a case existed — and everything downstream depended on that. Chasing works better when the customer recognises the reference. Reverts get answered when the context is already established. And the reverts population becomes interpretable, because a non-response now means something.

The general lesson is worth carrying: when a process depends on a counterparty responding, check first whether they know they are expected to. It is the cheapest thing on any operations backlog and it is almost never at the top of it.

Four rules for making a fragmented servicing estate measurable

- | | | |
|----|--|--|
| 01 | Separate the clocks before building the dashboard | A measure that mixes time you control with time you do not cannot be acted on and cannot be committed to. No visualisation recovers a conflated state model, and the state change is cheaper than the dashboard. |
| 02 | Make the clock stop on an event, not a status | A stoppable clock invites being stopped. Tie it to the revert actually being sent, report both populations side by side always, and treat a falling turnaround beside a rising reverts backlog as a warning. |
| 03 | Check the counterparty knows they are expected to act | The largest single cause of stalled cases was customers unaware a case existed. Where a process depends on someone outside the organisation responding, that is the first thing to verify and the cheapest to fix. |
| 04 | Look for the pathway that skips the automation | Channel-raised cases were acknowledged; internally raised ones depended on someone remembering. Two populations behaving differently and reported identically is a pattern worth actively hunting in any estate. |

The visibility work made the estate legible. The acknowledgement made it work. It is worth remembering which of those two was the smaller piece of engineering, and which one the operation actually noticed.